

COSC 39: Theory of Computation

Problem 1. Punched-tape machine

Punched-tape machine is a Turing machine equipped with an infinite two-way single-row tape, where each cell can be either punched or unpunched. Furthermore, once a cell is punched, it can never be unpunched. Alternatively, a punched-tape machine is just a Turing machine with unary alphabet consisting of a single symbol $\blacksquare$ (together with the empty cell symbol $\square$), where any symbol $\blacksquare$ written on the tape can never be erased. Prove that the punched-tape machine is equivalent in power to the standard Turing machine. In other words, the languages accepted by the punched-tape machines are exactly all the computable languages.

Solution.

Let M be a punched-tape machine; our goal is to show that M can simulate a standard Turing machine M' with a two-way infinite tape and the binary alphabet. We assume that $\Sigma' = \{0, 1\}$. Note that the converse holds immediately since every punched-tape machine is already a Turing machine.

The idea is to maintain a sequence of encodings $S_0, S_1, S_2, \dots$ of M' after k steps. Doing this allows us to never have to read or modify a previous encoding, letting us bypass the restriction imposed by the punched-tape machine.

Data Representation:

Each encoding S_k encodes a configuration (q, tape, h) of M' in two pieces:

1. A block of $|Q'|$ consecutive cells, where cell i is $\blacksquare$ if and only if the current state is q_i . Exactly one cell is $\blacksquare$ in a given encoding.
2. At step k , M' head has moved at most k positions in either direction. So for each position $p \in \{-k, \dots, k\}$ we use three consecutive cells on the punched-tape machine to encode both the tape symbol at p and whether the head is located at p . The encoding is as follows:

$\square\square\square = (\square, \text{head not here})$

$\blacksquare\square\square = (0, \text{head not here})$

$\blacksquare\blacksquare\square = (1, \text{head not here})$

$\square\square\blacksquare = (\square, \text{head here})$

$\blacksquare\square\blacksquare = (0, \text{head here})$

$\blacksquare\blacksquare\blacksquare = (1, \text{head here})$

Transition Simulation:

We start by encoding S_0 , i.e. the initial configuration of M' , once it's done the head returns to the start of S_0 . At step k , M runs a sub-routine that does the following:

1. Sweep S_k to find the current state q , the head position h (the unique cell in the tape section with third cell ■), and the symbol $a = \text{tape}[h]$.
2. Apply $\delta'(q, a) = (q', b, \Delta)$, since it is encoded in S_k .
3. Write S_{k+1} to the fresh tape following S_k , we use seven consecutive ■s as delimiters since that is the minimum that cannot naturally occur in any encoding.
 - **State:** Punch only cell q' to ■.
 - **Tape:** For each position $p \in \{-(k+1), \dots, k+1\}$, copy the symbol from S_k (or □ for the two new boundary positions), writing symbol b at $p = h$ and setting the head marker ■ at $p = h + \Delta$.
4. Move M 's head to the start of S_{k+1} and repeat.

M accepts if and only if the state corresponding to q' in M' is an accept state.

Thus we can simulate a standard Turing machine using a punched-tape machine.